

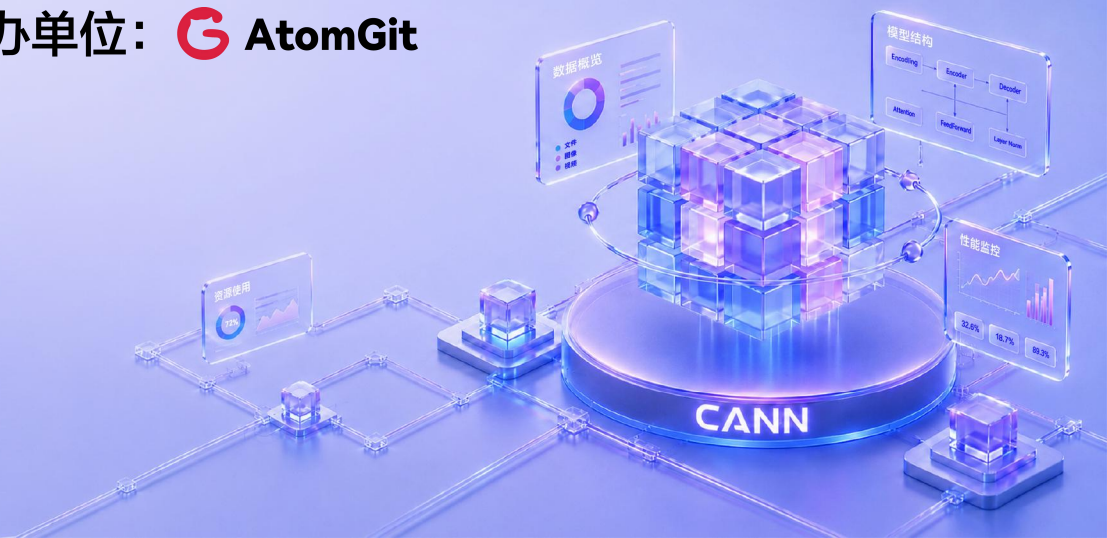
CANN Tech Connect 2026

CANN开发者Meetup

2026.8.8 中国·杭州

主办单位：CANN开源社区

协办单位： AtomGit





GLM-5.2 KV Cache Offload 系统优化实践

孔昊



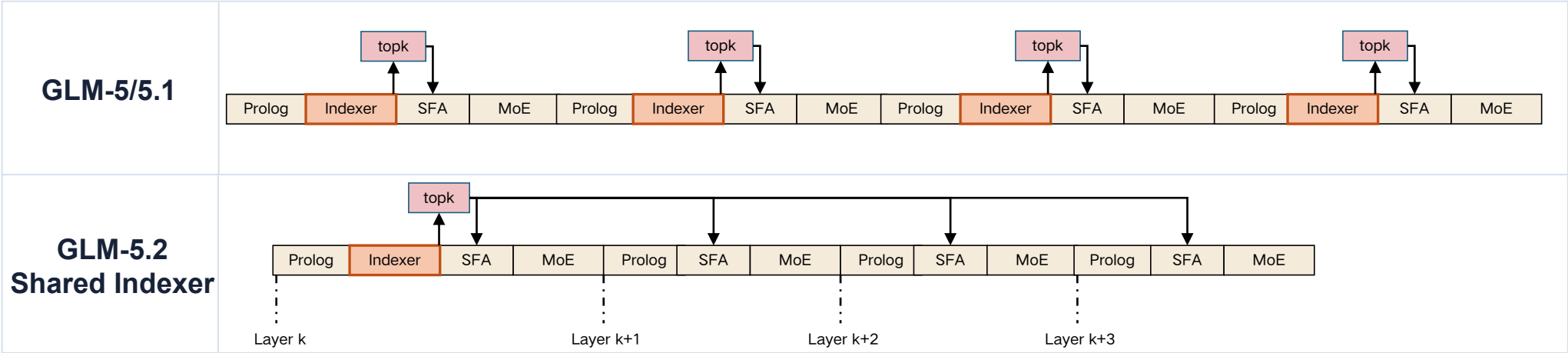
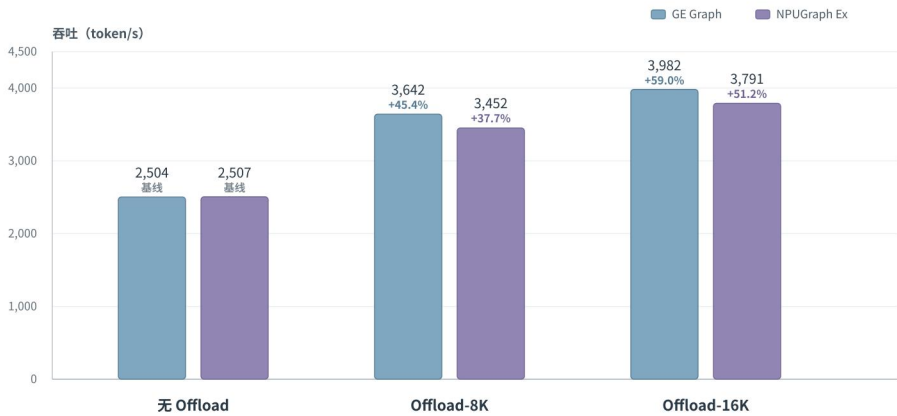
基于 GLM-5.2，使用 KV Offload 技术实现 50% 推理吞吐提升

753B / A40B
MoE 总参数 / 激活参数: 256 routed experts, Top-8

DSA + MoE
Lightning Indexer Top-K = 2048; latent KV rank = 512

- GLM-5 和 5.1 中，每个 Attention 层都有自己的 Indexer，计算本层的 Top-K。
- GLM-5.2 的 Shared Indexer 机制：四层一组，只有第一层有 Indexer，四层复用这个 Indexer 计算得到的 Top-K。
- Shared Indexer** 结构将在 Offload 方案中起到重要作用。

Offload 释放出的 Device 内存扩大并发，batchsize 从 64 提升到 128 吞吐净增约 50%



样例源码链接



content

目录

01

DSA 与 Offload 思想

全量保存与 Top-K 消费；Prefill 批量下沉，Decode 按需取回。

03

Shared Indexer Offload 方案

Prefill/Decode Offload 实现。

02

DSA 的 topk 重复特性与缓存池设计

先观察相邻 Step 的 Top-K 重叠，再扩大为 8K/16K Resident Pool。

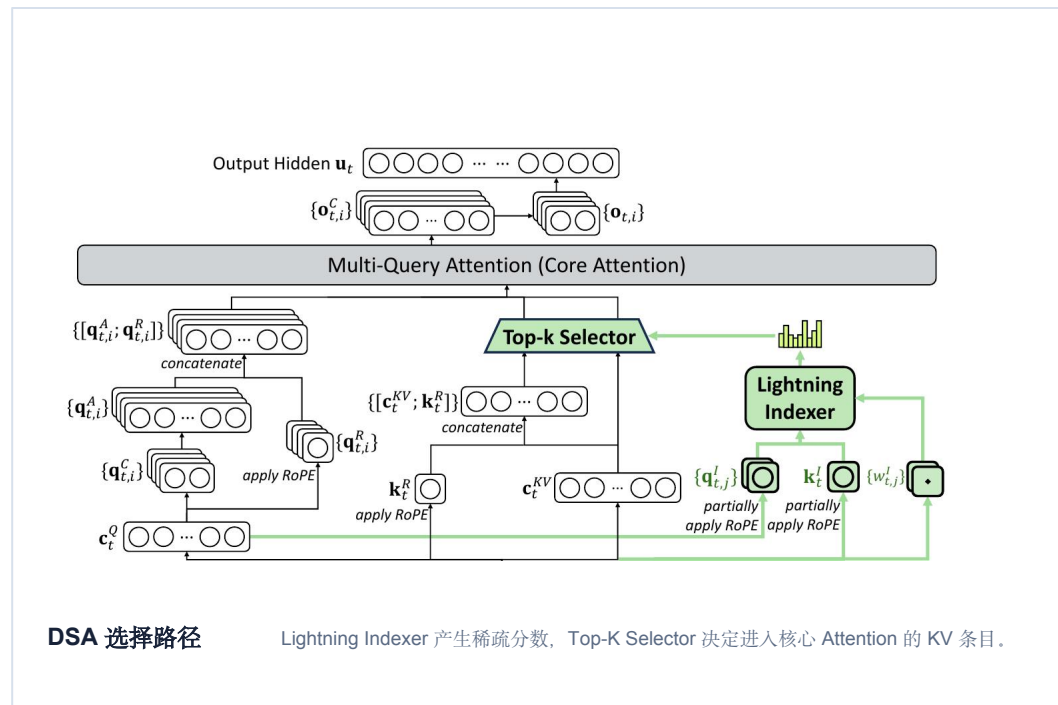
04

服务化框架适配

Vllm 框架 PD 分离适配。

DSA 特性: 保存全量历史, Attention 只消费 Top-K

Lightning Indexer 从完整历史中选择 2048 个位置, DSA 只读取这些位置对应的 KV。只有这 2048 个位置参与 Attention 计算。



Stored KV state
 $\propto \text{Context} \times \text{Batch} \times \text{Layers} \times \text{KV width}$

全量历史状态 N

1,048,576

配置值 · max position

单步选择 K

2,048

配置值 · index_topk

全量保存

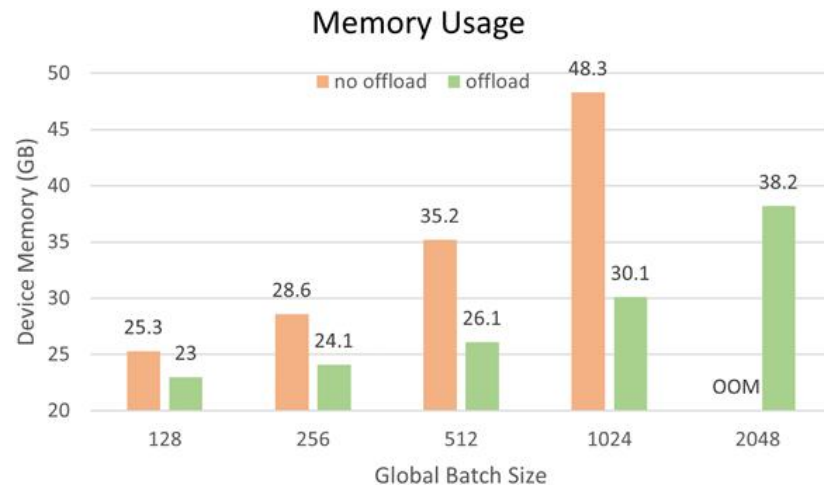
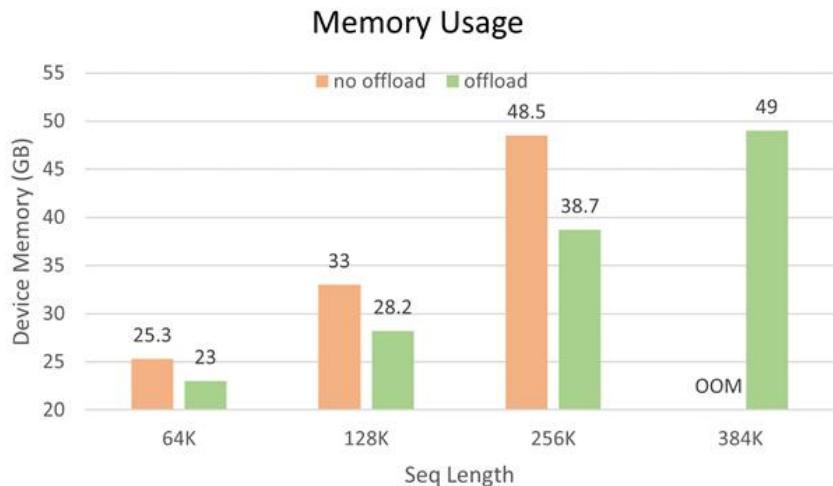
- 每个历史 token 都保留 MLA KV
- 未来 Query 可能访问任意位置
- 状态规模随上下文和并发量线性增长

稀疏消费

- Indexer 面向完整历史打分
- 输出当前 Query 的 Top-K IDs
- SFA 只读取对应的 K 条 KV

- 没有选中的 KV 仍然需要储存, 因为后续 Query 可能再次选中其中任意位置。因此 DSA 缩小了每步参与计算的数据量, 但完整历史仍然随序列长度增长。
- 两个层级的数据: 一是完整历史, 第二是实际参与计算的 2048 个位置。

KV Offload: 将 KV cache 卸载到 Host 侧



LLM 长序列推理中，KV Cache会随上下文长度呈线性扩张，成为内存和计算瓶颈，制约推理吞吐量提高。

Sparse Attention: 利用注意力层常有的稀疏特性，每步迭代中，当前 token 只关注序列中部分重要的 token，提高计算效率

KV Offload: 将 KV cache 卸载到 Host 侧，解码时按需访问，Device 侧内存得以释放，可以增大 batch size 进而提高吞吐量

特性	计算效率	内存效率	吞吐提升
稀疏注意力	√		
KV Offload		√	
两者	√	√	√

Offload 思想如何应用到 DSA 结构: Prefill 阶段卸载, Decode 按需取回

Prefill: 在 Prolog 之后, 把 KV Cache 完整搬运到 Host 侧。



阶段	数据方向	传输特征
Prefill	Device → Host Full KV	完整、大块、一次性下沉
Decode	Host → Device Selection	Top-K 驱动、稀疏、每步发生

Decode: Indexer 先产生 Top-K ID, 把对应的 KV 取回 Device, 给 Attention 计算。



新的系统问题: 单步搬运量从 N 降到 K, 但稀疏 H2D 会在每个 Decode Step 重复发生; 相邻 Step 的数据复用决定实际 H2D开销。

- Offload 额外开销: Top-K KV 的 H2D 搬运。



使用 DSA 相邻 Step 的 Top-K 重复特性构建缓存池，减少搬运开销

对于 DSA 的 topk 选择，对当前step， topk 重要的 token，在接下来的 step 中也大概率重要，因此**相邻 decode step 的 topk 重复率很高**

同一句子中的 Top-K 位置与连续生成

Base prompt: The quick brown fox jumps over the lazy dog.



相邻 Step 的集合关系

重叠 58.85%

新增 41.15%

典型输入场景相邻 Top-K 轨迹实测结果

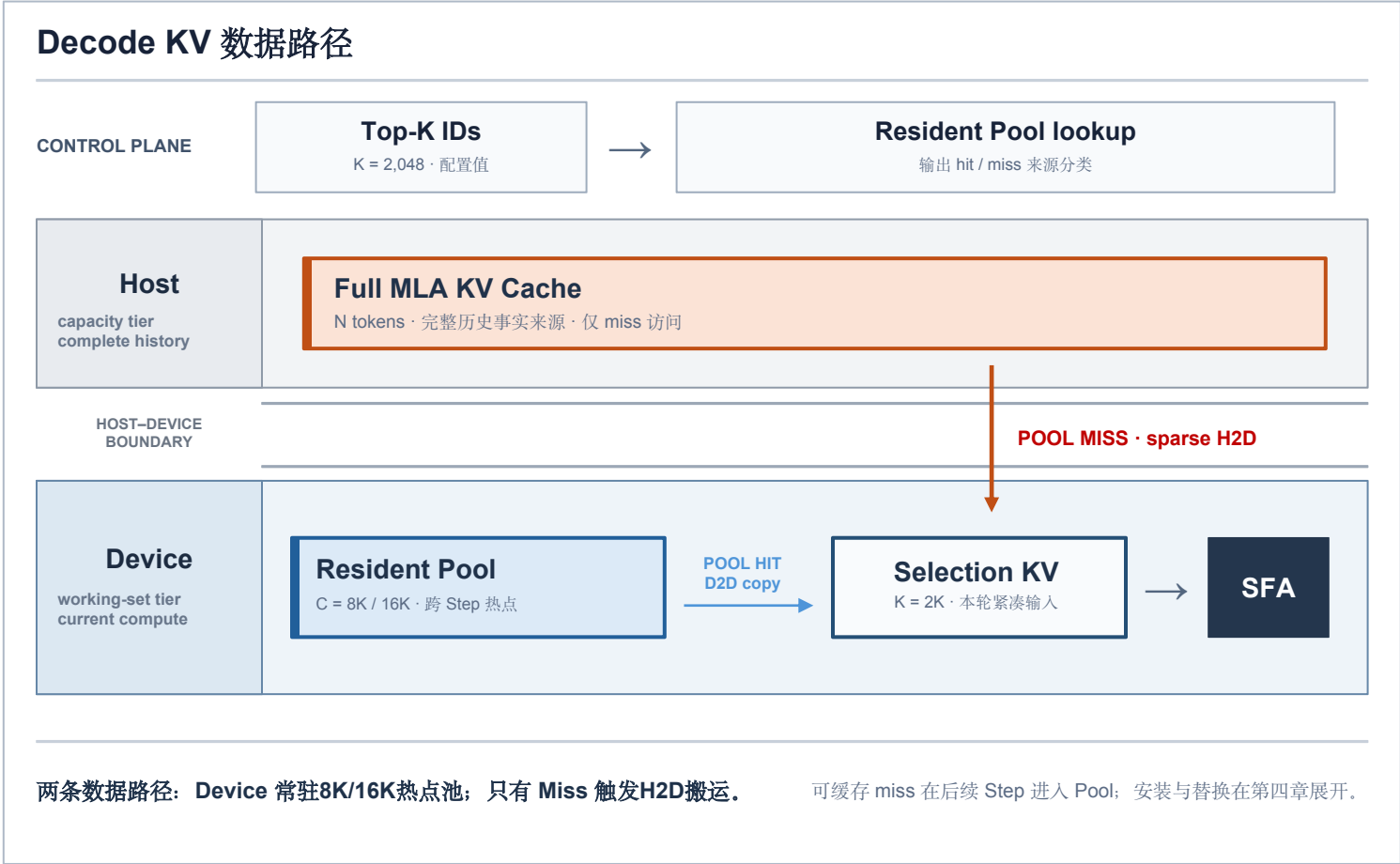


■ 重叠率随 Layer、Step 与语义主题变化

利用这个特性，可以在**Device 上构建一个 Top-K 大小的缓存池**。把每一步 Top-K 对应的 KV Cache 留在 Device，在下一步，命中缓存的不需要再进行 H2D 搬运。

扩大缓存池，提高缓存命中率，瓶颈从搬运转移到缓存池维护

更进一步，**专门做一个 Device 上的更大缓存池**。命中缓存的做低成本 D2D 搬运，未命中的做 H2D 搬运。



对象尺度与职责

N	>>	C	>	K
Host Full KV		Resident Pool		Selection

扩大 Device 复用窗口

保留范围	Hit	Host miss
上一步 2K Selection	58.85%	41.15%
8K Pool	90.50%	9.50%需要搬运

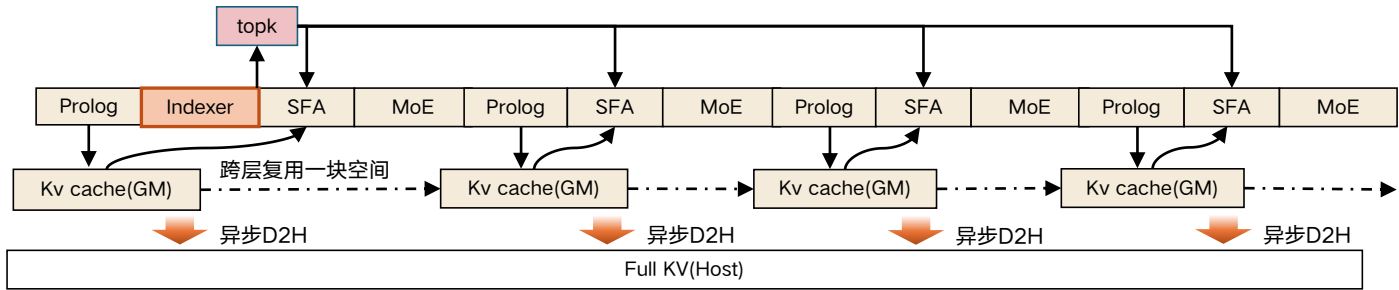
把缓存池大小扩大到 8K，缓存命中率提升到90%

瓶颈开销变化

- **问题:** 引入热点池，提高命中率后，H2D搬运耗时减少，**查询命中&缓存池更新耗时瓶颈，scalar-bound**
- **办法:** 利用GLM-5.2 shared indexer的特性，**摊薄查询命中的成本**
- **Install操作无数据依赖，放在测流与Attention/MoE并行**

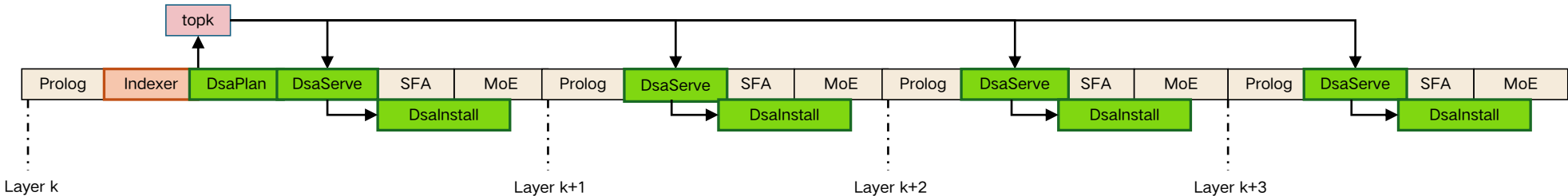
根据缓存池维护瓶颈设计 Offload 方案

Prefill

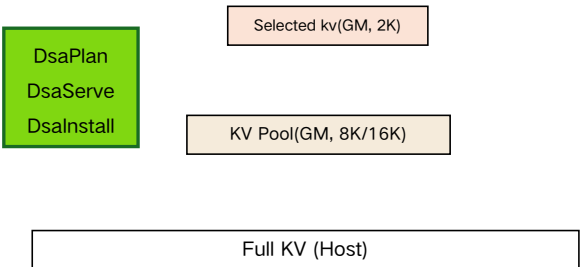


- 当前层产生的 KV 临时保存在 Device，随后异步写入 Host。Device 上是多层复用同一块临时空间，因此这里有一个异步依赖，每层传输完成后，这块空间才能给下一层使用。

Decode



- 第一层先通过 Indexer 产生 Top-K，接着依次执行 DsaPlan 和 DsaServe。Selection 准备好以后，主流继续执行 Attention 和 MoE，DsaInstall 则放到辅流执行。
- 后面的层复用第一层的 Top-K 和 Plan。每层的 KV 内容不同，所以各层仍然分别执行 Serve 和 Install。



PER INDEXSHARE GROUP · ONCE

DsaPlan: 查询命中

对共享 Top-K 做一次来源分类与 miss 规划，生成四层共同复用的 Plan。

每四层执行：判断selected token位于host还是device。

PER LAYER · CRITICAL PATH

DsaServe: 搬运

依照共享 Plan，从本层 Pool 或 Host Full KV 准备本层 Selection。

每层执行：将selected token 做H2D/D2D搬运。

PER LAYER · SIDE STREAM

DsaInstall: 更新缓存

把可缓存 miss 回填到本层 Resident Pool，淘汰很久不用的KV。

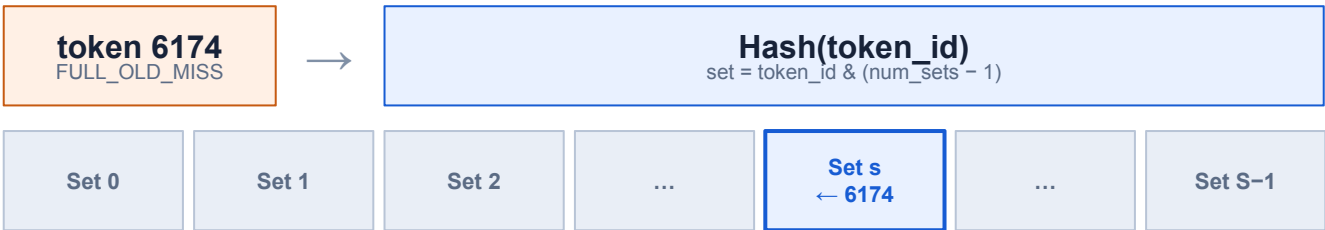
每层执行：更新Device上的缓存池。

缓存池的维护——哈希表查询，FIFO更新

- 查询命中：用哈希表给整个缓存池分桶，只和桶内逐元素比较。
- 缓存更新：每个哈希桶内用先进先出轮转更新。

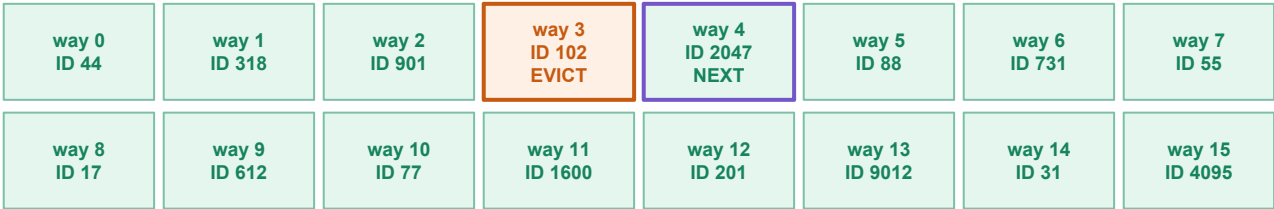
缓存更新机制

示例：miss token 6174 的安装位置



Set s · 16 Ways

Hash 只选 Set; 替换仅发生在该 Set 内



一次 FIFO 安装

- Hash 定位 Set s
- counter 选择 way 3
- 清理 way 3 的旧 token 映射
- 写入 token 6174 的各层 payload
- counter 从 3 推进到 4

Pool	Ways	Sets
8K	16	512
16K	16	1024

完整模型实测：Batch × 2，单步延迟增加约 30%，吞吐净增约 50%

HARDWARE 双节点 Atlas A3 16 卡	MODEL GLM- 5.2- W8A8	CONTEXT 64K · MTP3	BATCH 64 → 128	EXECUTION GE Graph NPUGraph Ex
----------------------------------	-------------------------	-----------------------	-------------------	--------------------------------------

完整六组端到端结果

THROUGHPUT · token/s

执行模式	配置	Batch	Pool	延迟	吞吐 2.7	吞吐 实际
GE Graph	No Offload	64	—	71.21 ms	2,427	2,504
GE Graph	Shared Offload	128	8K	100.55 ms	3,437 +41.6%	3,642 +45.4%
GE Graph	Shared Offload	128	16K	92.03 ms	3,755 +54.8%	3,982 +59.0%
NPUGraph Ex	No Offload	64	—	71.59 ms	2,414	2,507
NPUGraph Ex	Shared Offload	128	8K	105.04 ms	3,290 +36.3%	3,452 +37.7%
NPUGraph Ex	Shared Offload	128	16K	95.67 ms	3,612 +49.6%	3,791 +51.2%

系统结果摘要

CAPACITY CHANGE Global Batch · +100%	64 → 128
STEP LATENCY · 16K GE / NPUGraph Ex · 推导值	+29.2 / +33.6%
ACTUAL THROUGHPUT · 16K GE / NPUGraph Ex · 实测	+59.0 / +51.2%

Offload 算子耗时

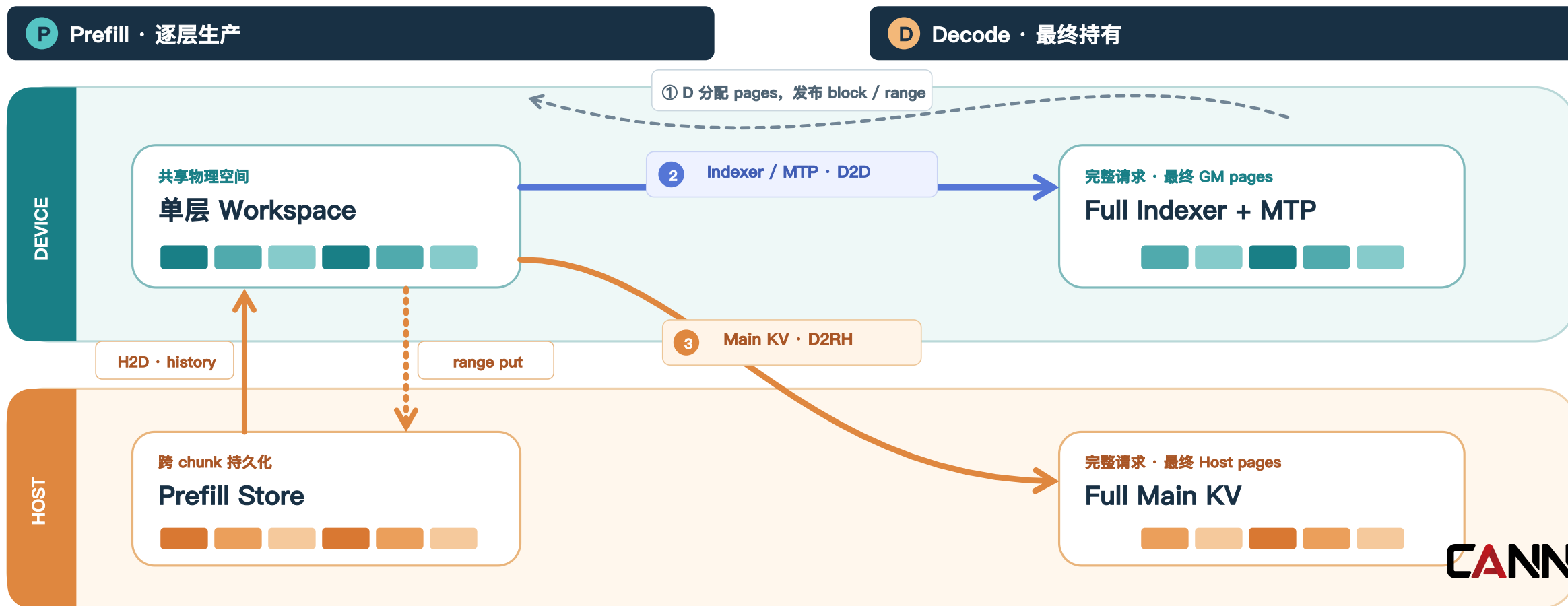
DsaPlan	≈20	μs
DsaServe	≈37	μs
DsaInstall	≈9	μs



将 Offload 应用到服务化框架 — PD Cache 传输

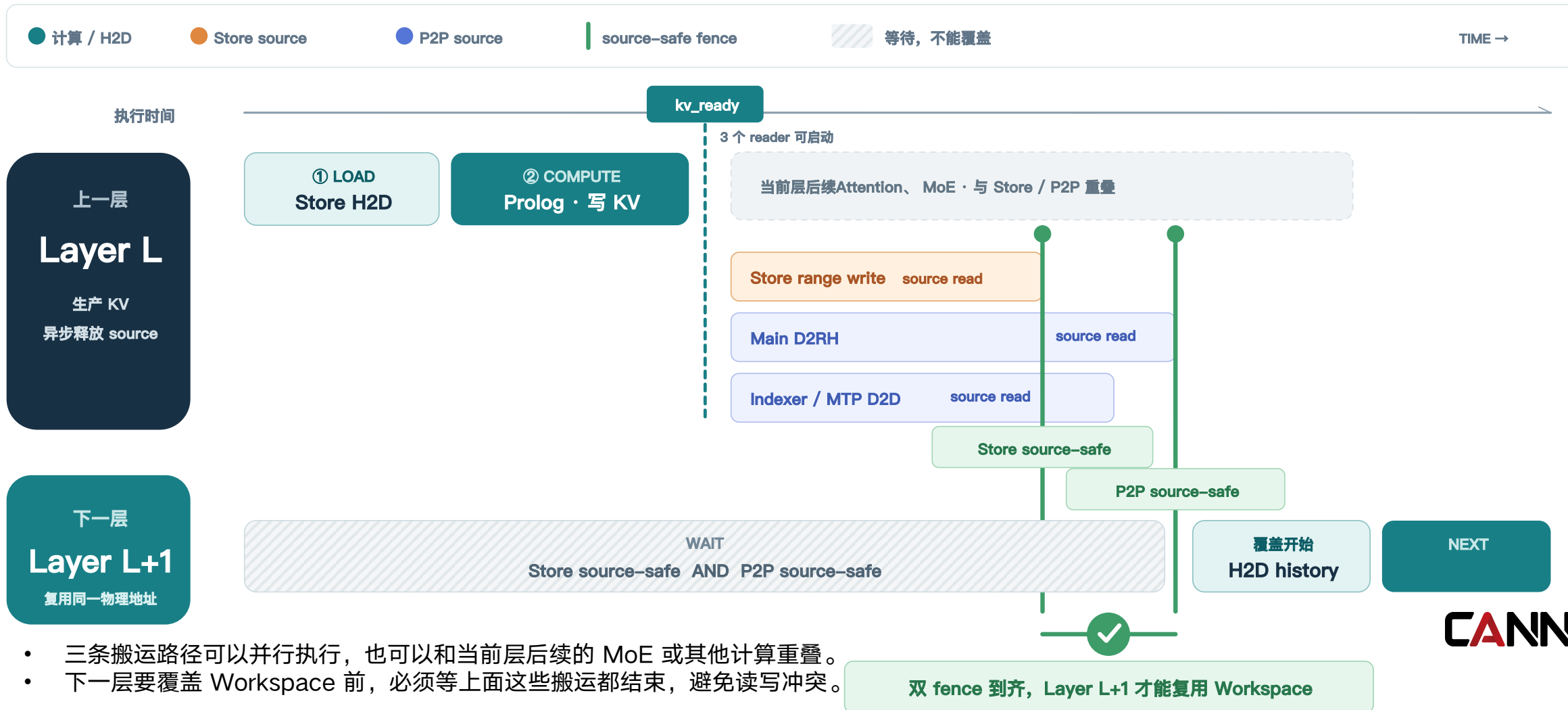
将以 vLLM 为例，完成 KV Offload 在服务化推理框架中的落地，并将相关实现开源至 `cann-recipes`。

- **Prefill** 计算前从 **Store** 恢复历史 KV，主要服务于 Chunked Prefill 和 Prefix Cache 命中。
- 当前层 Prolog 计算完成后，**Indexer/MTP Cache** 走 **D2D** 传输到 Decode 的 Device；**Main KV** 走 **D2RH** 从 Prefill Device 直接写到 Decode 侧的 Remote Host。



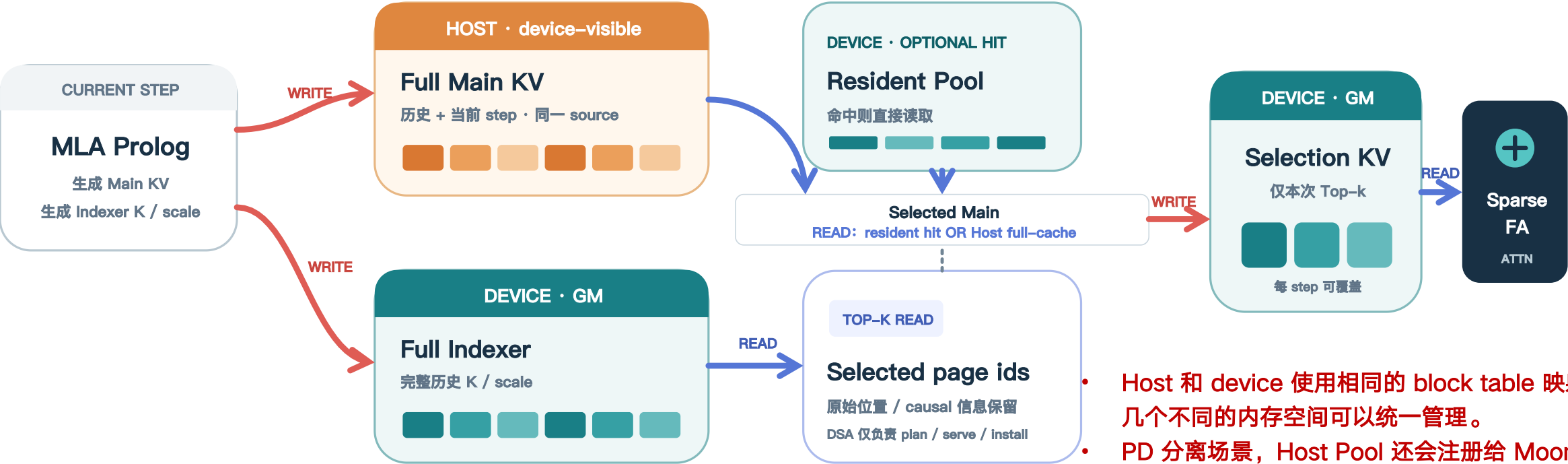
服务化框架适配 — Prefill Timeline: 三条传输路径并行

- Prefill 的执行过程：先通过 Store H2D 把需要的历史 KV 搬到 Device Workspace。随后执行当前层 Prolog，并把新生成的 KV 写入同一个 Workspace。
- Prolog 完成并产生 kv_ready 后，会触发三条搬运路径：KV 写入自己的 Host，Main KV 发送到 Decode 机器的 Host，Indexer KV 和 MTP KV Cache 发到 Decode 机器的 Device。



服务化框架适配 — Decode DataFlow

- 1 当前 step 写入完整 cache
- 2 Indexer 选页, Main 按需读取
- 3 组装 selection, 交给 FA



- Host 和 device 使用相同的 block table 映射。这几个不同的内存空间可以统一管理。
- PD 分离场景, Host Pool 还会注册给 Mooncake, 用于接收 Prefill 侧传输过来的 Main KV。





Thanks!

cann-recipes-infer 样例源码



cann-recipes 交流群



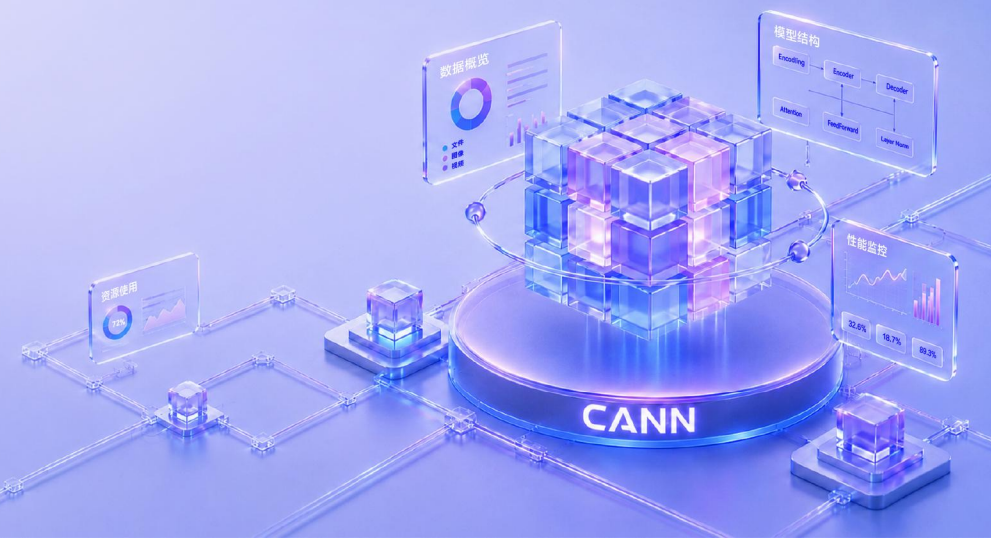
GLM5.2 Offload 优化源码已开源至 cann-recipes-infer，欢迎广大开发者体验并参与贡献！





Agent 赋能模型优化：Hy3 昇腾推理实践

王子恒



目录

Part 1 Hy3 解析与模型 Agent 体系介绍

Part 2 结合 Agent 优化的 950PR 集群部署解析

Part 3 优化总结与 Future Plan

模型解析 —— Hy3 整体结构

模型规格

- 支持 256K 上下文;
- GQA 多头注意力架构;
- 192 专家 MoE 架构 top-8 稀疏激活;

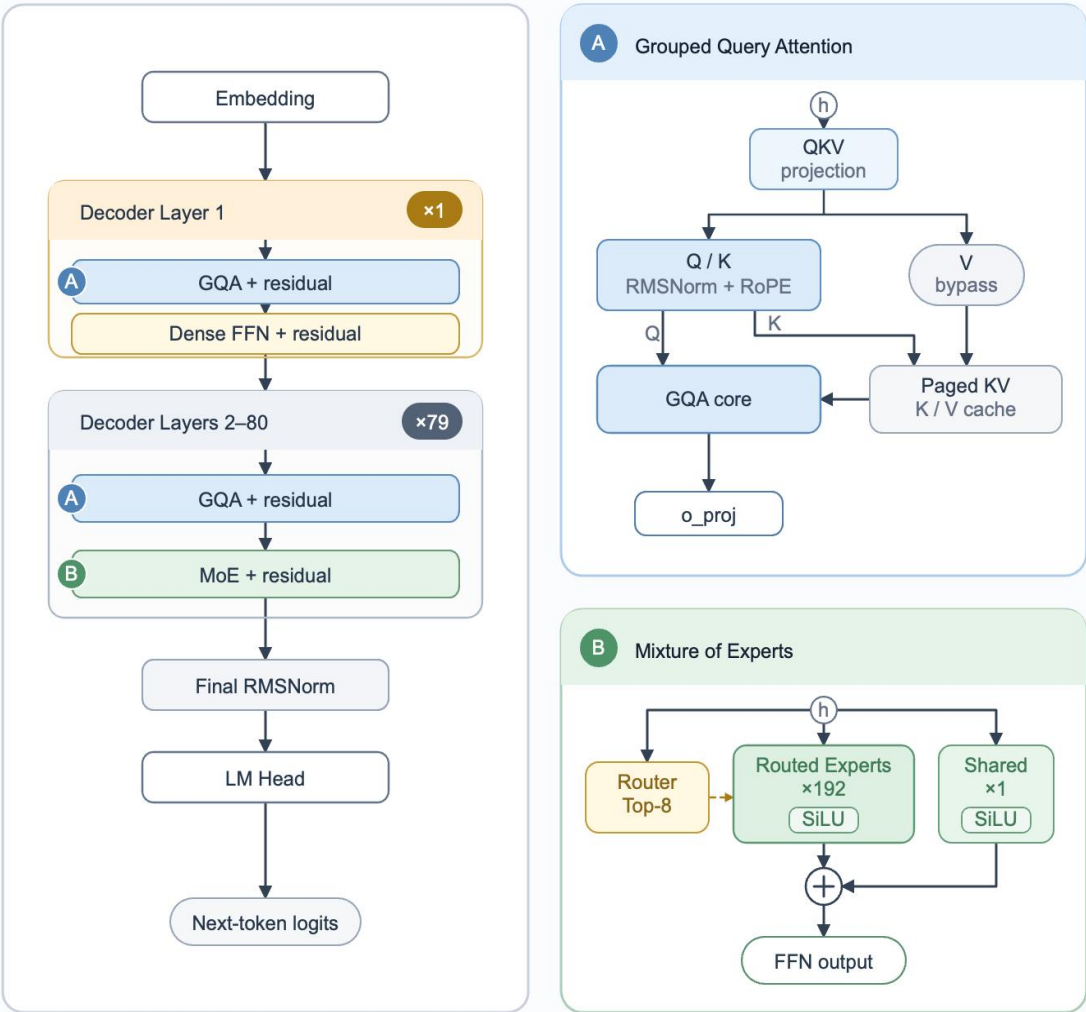
架构	稀疏 MoE	总参数 / 激活	295B / 21B
层数	80 层 (第 0 层 Dense, 1-79 层 MoE) + 1 层 MTP	隐藏维度	4096
注意力	GQA: 64 个 Q 头 / 8 个 KV 头, head_dim 128	专家	192 个, 激活 top-8, 1 个共享专家

上下文	256K	词表	120832
-----	------	----	--------

■ no_think 极速响应 ■ think_low 快速思考 ■ think_high 深度推理

部署场景: 昇腾 950 PR/DT 4 die & 昇腾 Atlas A3 16 die
80% 方案设计+代码开发可由 Agent 完成

模型结构

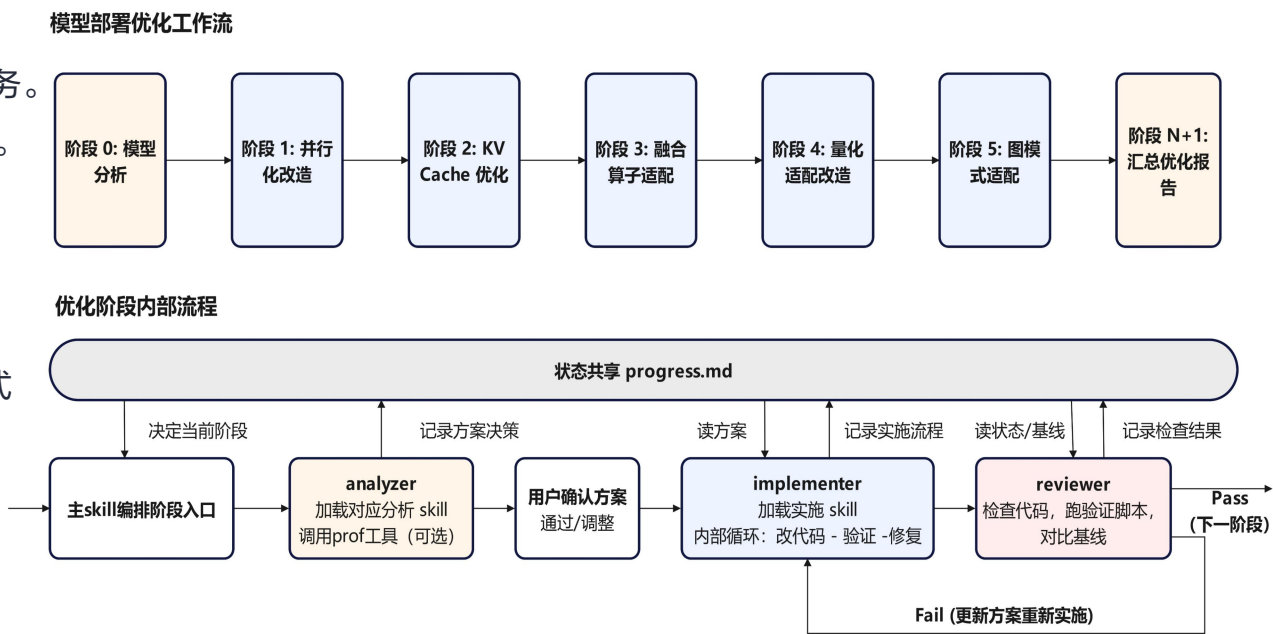


模型 Agent 工作流 —— 推理部署优化

模型 Agent 工作流的核心价值：开发效率提升、使用门槛降低

模型基础部署优化

- 为什么要做模型 Agent 体系？
 - 问题：通用 Agent 难以稳定完成长链路、高复杂度的 NPU 推理优化任务。
 - 方案：把优化经验与样例沉淀为可执行、可验证、可复用的 Skill 工作流。
- Agent 端到端工作流 —— 稳定实现基础部署优化闭环
 - 覆盖模型迁移适配、并行切分、KVCache 优化、融合算子替换、量化、图模式适配、debug 调试等 CANN 原子化优化特性，支持新模型一键式 NPU 部署优化。
 - 通过完善的 Harness 约束机制，包括 Agent team 角色分工、Progress 状态同步、Hook 约束和验证门禁，确保 Agent 稳定高效完成优化流程。



模型进阶性能调优

- 工作流驱动的人机协同 —— 进阶优化追求极致性能
 - 通过profiling 采集与性能分析拆解 Skill，支持拆分模块/层耗时与抖动等性能瓶颈识别呈现，为优化项提供证据支撑。
 - 利用多流、SuperKernel、Prefetch 等进阶优化 Skill，使能优化特性进一步提升推理性能。



模型 Agent workflow —— AMCT模型量化

AMCT 是 CANN 提供的模型量化压缩工具，支持 Agent 自动化量化权重和方案导出

整体流程

- 使用场景：对原始权重希望得到昇腾可直接部署的量化权重；
- 方案选择：检查可用性与是否已有参考方案，决定量化方式；
- 量化实施：借助Quant Tools进行量化实验，导出可部署量化权重与方案介绍；
- 交付沉淀：方案与轨迹更新为casebook，形成后续复用资产；

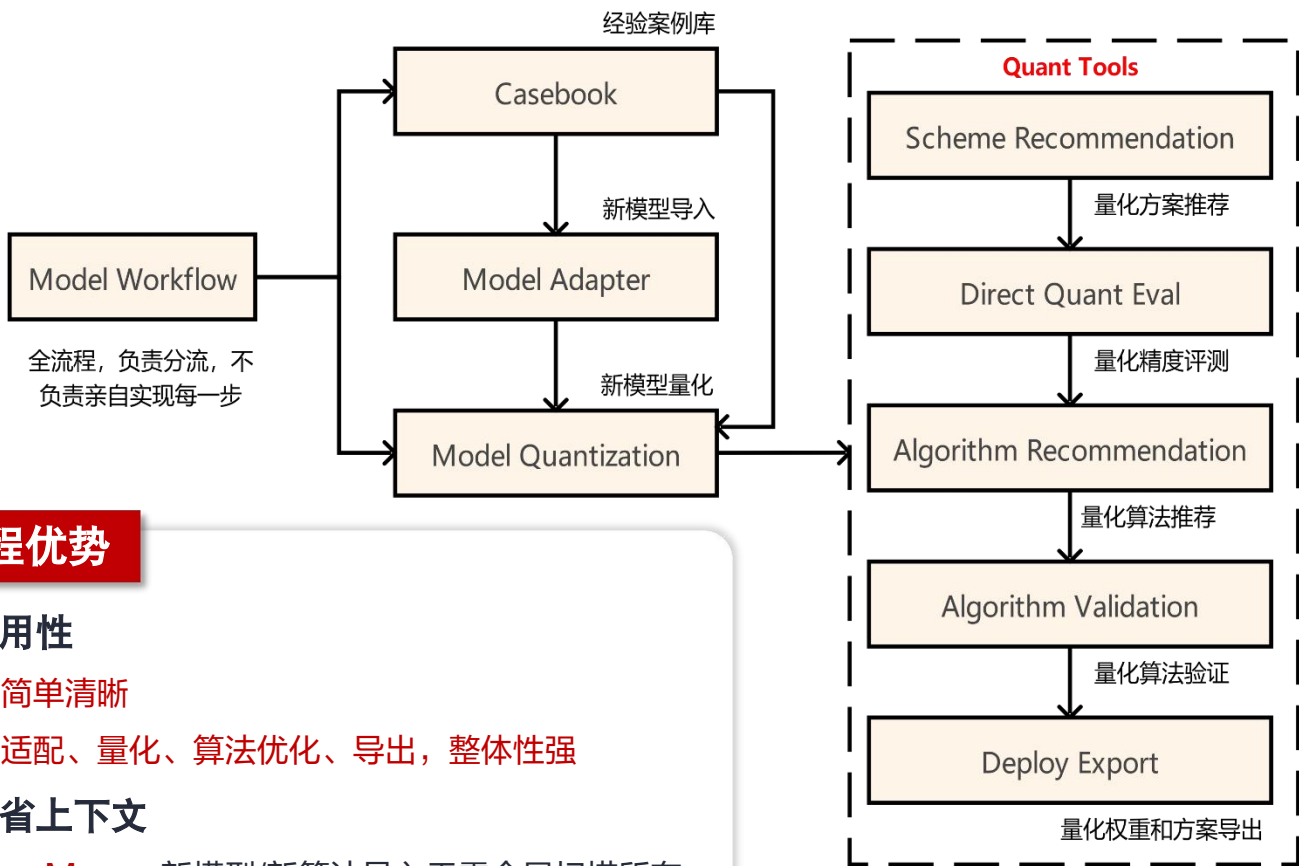
模块介绍

- Model adapter(只做新模型接入)
 - 模型注册
 - BF16 基线测试与关闭量化后的浮点等价测试
 - 最小PTQ冒烟
- Model quantization(量化阶段总控)
 - 量化方案推荐：根据Casebook给出量化方案，或用户指定
 - 直转量化评测：基于已有量化方案做直转量化测试
 - 量化算法推荐及验证：根据Casebook推荐量化算法，并验证
 - 量化权重导出：导出量化部署权重及方案说明文档

流程优势

- 易用性
 - 流程简单清晰
 - 集合适配、量化、算法优化、导出，整体性强
- 节省上下文
 - Repo-Map：新模型/新算法导入无需全局扫描所有函数，对关键锚点文件扫描，节省上下文，同时了解工具整体框架

项目	内容
最近校验日期	2026-04-17
使用方式	先读 repo-map -> 抽查锚点 -> 只更新受影响 section
适用场景	新模型适配与 PTQ 流程定位



目录

Part 1 Hy3 解析与模型 Agent 体系介绍

Part 2 结合 Agent 优化的 950PR 集群部署解析

Part 3 优化总结与 Future Plan

模型 Agent workflow —— 并行策略分析

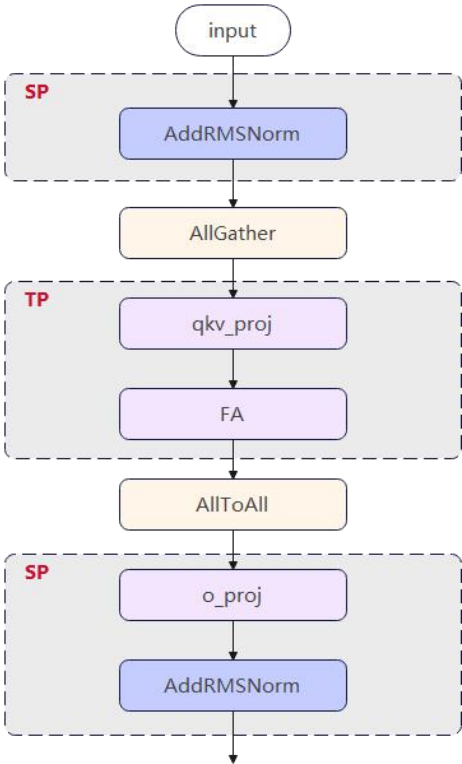
部署场景：昇腾 950 PR/DT 4 die

➤ Agent 分析与实施：参考 `model-infer-parallel-analysis / -impl`，先分析模型结构与硬件规格，对照最佳实践部署样例，分析计算、显存、通信，确定整体并行策略；再落地切分、搭建各通信组、按维加载权重，并验证多卡输出语义正确。Agent 实现基础部署方案，人机协同优化通信方式追求极致性能。

Prefill Attention SP+TP

➤ Attention部分可以考虑的并行方式包括DP/TP/SP，由于TTFT的时延要求，Attention的并行策略主要考虑TP/SP。Hy3模型为GQA结构，因此TP并行是比较自然的选择。在TP的基础上对FA前后的算子增加SP并行，进一步减少整体时延。

- FA前通信：使用AllGather将各Rank的子序列聚合成完整序列，配合FP8/MXFP8量化减小通信数据量。
- FA后通信：使用AlltoAll将各Rank的注意力输出按序列维度重新分发，相对于纯TP的ReduceScatter进一步减少通信耗时。



MoE EP

➤ MoE部分支持TP和EP两种并行方式。TP并行会将GMM的K或N轴切得过小，导致计算性能下降。EP并行每个Rank持有部分专家，通过InitRouting完成token与专家的路由匹配，在ep_size数较小时通信开销可控。

EP并行进一步分为AG-RS (AllGather-ReduceScatter) 方案和Double Routing (AlltoAll) 方案：

EP 方案	通信算子	适用场景
AG-RS	AllGather + ReduceScatter	小 EP: top_k > ep_size
Double Routing	AlltoAll × 2	大 EP: ep_size > top_k

■ 当前模型top_k=8，在ep_size=4时top_k > ep_size，AG-RS方案通信数据量更小，为默认优选方案。



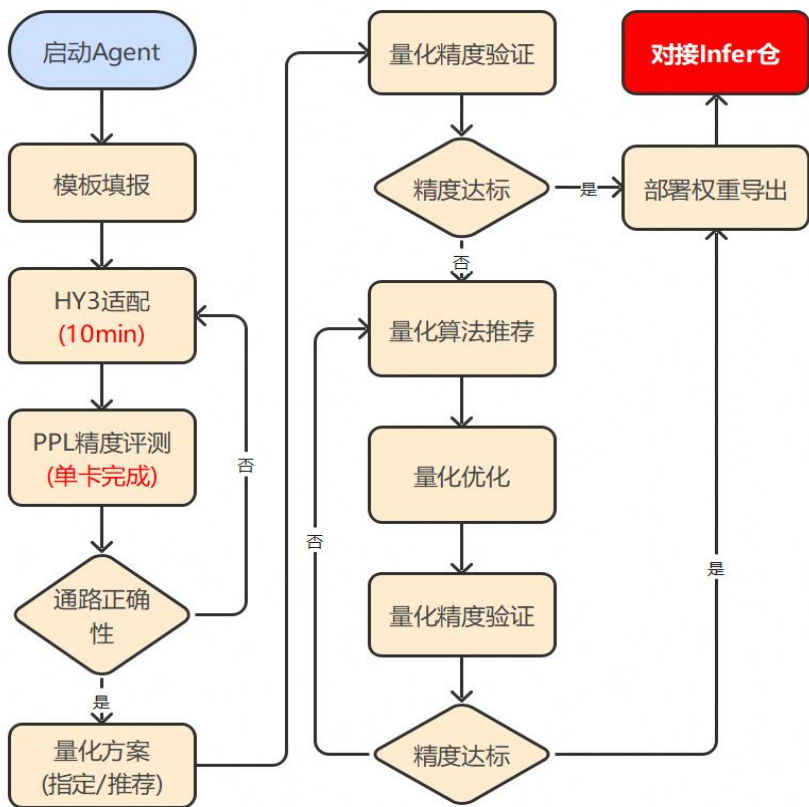
模型 Agent workflow —— AMCT & 量化方案实施

AMCT 量化仓与 infer 推理仓联动：生成量化权重/方案 → 对接交付产物 → 模型量化接入

AMCT量化权重/方案 workflow

量化格式差异

950PR/DT系列芯片基于新增支持业界标准FP8/MXFP8/MXFP4等低数值精度数据格式MX (纯cube), fp8 (CV mix), MX更方便做多流并行, 计算和搬运的开销都会更小, 在计算bound的场景, 对比FP8收益明显。



模型量化接入 workflow

Agent 分析与实施：参考 **model-infer-quantization**, 先做量化初评估、核对量化产物契约, 再建立 runtime 映射、按参数属性加载量化分片进去, 识别接入量化算子。

模块	量化档	说明
Attention qkv_proj / o_proj	W8A8 (MXFP8)	权重、激活均 8 位
KV Cache	C8	QK 动态 Per-Token-Head FP8; V 动态 Per-Head FP8; Scale 为 FP32
MoE 路由专家	W4A8 (MXFP4)	权重 4 位、激活仍 8 位, 算量占比最大
MoE 共享专家	W8A8 (MXFP8)	—
LM Head	不量化	保持输出层精度

FP8/MXFP8 W8A8 + MoE W4A8推理

基于950PR/DT平台, 支持了**原生FP8量化**, 同时也支持了使用**MXFP8替换原生FP8格式的Linear模块**, **MOE部分替换MXFP4**, 在Prefill和Decode高吞吐场景可以进一步提升算力利用率, 提高整体性能。

模型 Agent 工作流 —— KV Cache & 融合Kernel & 多流

KV Cache

- Agent 分析与实施: 参考 [model-infer-kvcache](#), 把 KV 管理从模型自管缓存改造为框架公共分页管理, 接入 FA 融合算子, 构造 block_table 与 slot_mapping。

KV Cache C8 量化

QKV 前处理算子在核内完成 K/V 动态 FP8 量化并写回 C8 KV Cache 与 k_scale, FP8量化 GQA FA 算子再利用该 C8 Cache 做全量化注意力。

多流优化

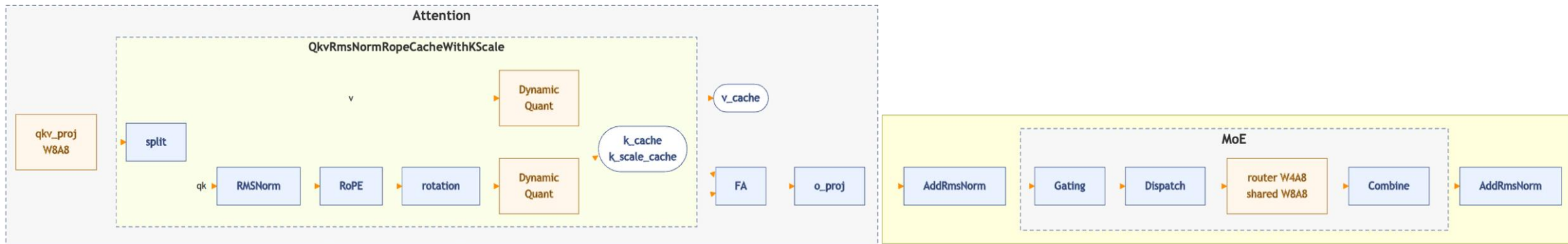
- Agent 分析与实施: 参考 [model-infer-multi-stream](#), 做整网 DAG 与模块间并行性分析, 识别无数据依赖的可并行分支, 派生数个多流编排方案并用 Profile 验证最优 overlap 方案真实生效。

共享专家&路由专家并行

- 路由专家: 计算与通信密集, 依次执行 Gating、路由专家路径与最后的 combine 相加; 共享专家: 耗时短算子少; 可以并行掩盖在路由专家下;
- 效果: 共享专家耗时完全掩盖, 路由专家性能不劣化。

融合算子适配

- Agent 分析与实施: 参考 [model-infer-fusion](#), 按模块识别Attention、MOE、Norm等可融合的计算模式, 按计算语义匹配并替换为 CANN 已支持的 torch_npu / AscendC 融合算子, 核对入参选择与前置改造项, 分组接入并验证与原实现对齐。



新增已开源融合Kernel: `qkv_rms_norm_rope_cache_with_k_scale`和GQA FP8全量化版本`fused_infer_attention_score`

当前新算子开发/接入仍需人工介入指导, 未来将联动算子生成 agent, 实现从自动融合范围分析到新算子开发的完整 Agent 流程。

模型 Agent workflow —— npugraph_ex 图模式 & MTP

npugraph_ex 图模式

npugraph_ex是一个基于CANN的PyTorch FX图优化组件，通过torch.compile入口无缝对接NPU图模式（ACLGraph），帮助开发者在零额外成本下充分释放NPU性能。

- **快速接入**：CANN提供ACLGraph，有效的消除了算子调度时延，并已经集成在PyTorch中，可快速使能；
- **高性能**：在复用原有ACLGraph调度能力前提下，借助于PyTorch的FX图，进一步叠加NPU亲和的优化；
- **可集成**：面向大模型推理的服务化框架，可以做到快速集成，无缝对接到vLLM/SGLang等服务化框架中。

➤ Agent 分析与实施：参考 **model-infer-graph-mode**，把 decode 路径适配到图模式后端（950 PR/DT用 npugraph_ex），定位并消除 graph break，接入编译缓存降低重复编译开销。

模型编译缓存

成图编译涉及两段耗时，一段是Dynamo的编译耗时，另一段是基于Dynamo编译出的FX图进行再处理的耗时。npugraph_ex提供了模型编译缓存方案，通过cache_compile接口**将首次编译的结果落盘**，从而加快torch.compile图模式的启动时间。



MTP 投机推理

MTP 投机推理用一个轻量 draft 头在一次前向里**预测后续多个token**，再由主模型校验接受正确的部分。Hy3 复用 checkpoint 中第 80 层的 MTP 权重，接入单层 MTP、next_n= 1~3 的投机解码。主 verify 模型与 MTP draft 头统一到请求维并行的 decode 切分路径。

➤ Agent 分析与实施：参考 **model-infer-mtp**，实现 MTP 头，包含MTP 层容器、主类前向、get_cache_info、权重映射与注册，并验证投机接受率与输出精度一致。

目录

Part 1 Hy3 解析与模型 Agent 体系介绍

Part 2 结合 Agent 优化的 950PR 集群部署解析

Part 3 优化总结与 Future Plan

模型 Agent 推理优化成果总结

整网优化总结

混合并行部署:

- Prefill SP+TP+EP;
- Decode DP+ TP+EP;
- Embedding/LM Head TP;

低比特量化

- Weight W8A8 FP8/MXFP8;
- MoE W4A8 MXFP4 ;
- FA 全量化FP8 C8;

融合Kernel

- FP8全量化版本fused_infer_attention_score
- 大融合算子: qkv_rms_norm_rope_cache_with_k_scale

通信优化

- MoeDistributeDispatch、MoeDistributeCombine、HCCL AIV 通信;

多流并行与控核

- MOE共享/路由专家多流;

NpuGraph_Ex 图模式

多头MTP

模型 Agent 优化效果

基础部署优化各阶段性能/耗时统计

部署场景: 昇腾 950 PR/DT 4 die

优化阶段	工作耗时	Decode 单步	相对基线
模型分析+部署骨架	~0.7 hr	—	—
AMCT量化方案	~0.5 hr	—	—
并行切分方案与实施	~2.2 hr	103.18 ms	1.0x
KVCache 与 FA 改造	~2.7 hr	96.40 ms	1.07x
融合算子适配	~2.5 hr	44.94 ms	2.30x
MXFP 量化接入	~1.2 hr	42.59 ms	2.42x
图模式适配	~1.0 hr	17.98 ms	5.74x
合计	~10.8 hr	17.98 ms	5.74x

80% 方案设计+代码开发可由 Agent 完成

Future Plan

Hy3 进阶性能优化

- **稀疏 Attention**: 加入Hy3 原生支持的 Stem 块稀疏注意力算法, 进一步降低时延;
- **双bf16**: topK 门控拆分多段低精度运算重组, 节省FP32 算力;
- **MegaMoe**: 采用 MegaMoE 大规模融合 Kernel, Attention Flash Decode 优化等;

模型 Agent 功能改进

- **反馈迭代**: 从实际部署中复盘易用性及断点, 持续优化 Skill 覆盖与Harness流程约束;
- **流程完善及规模扩展**: 新增各类高阶优化特性与最佳实践样例, 提高覆盖率与准确度;
- **性能分析能力**: 提高优化特性使能、Profiling 性能拆解分析 、定向调优的闭环效率;
- **新方案设计能力**: 完成新优化方案的分析和派生, 例如融合算子方案设计, 联动算子自动生成;

欢迎体验和贡献

cann-recipes-infer 代码



CANNBot 代码仓



AMCT代码仓



cann-recipes 交流群



cann-recipes-infer:

<https://gitcode.com/cann/cann-recipes-infer>

AMCT:

<https://gitcode.com/cann/amct>

CANNBot:

<https://gitcode.com/cann/cannbot-skills>



试用



提 issue



补充案例



参与社区讨论

欢迎广大开发者体验并参与贡献，如有疑问也可通过
issue、SIG或者cann-recipes交流群联系我们！

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

<https://gitee.com/cann>

08/08 CANN meetup 线下互动交流帖



参与趣味知识问答，赢取免费算力或者
CANN周边礼品！

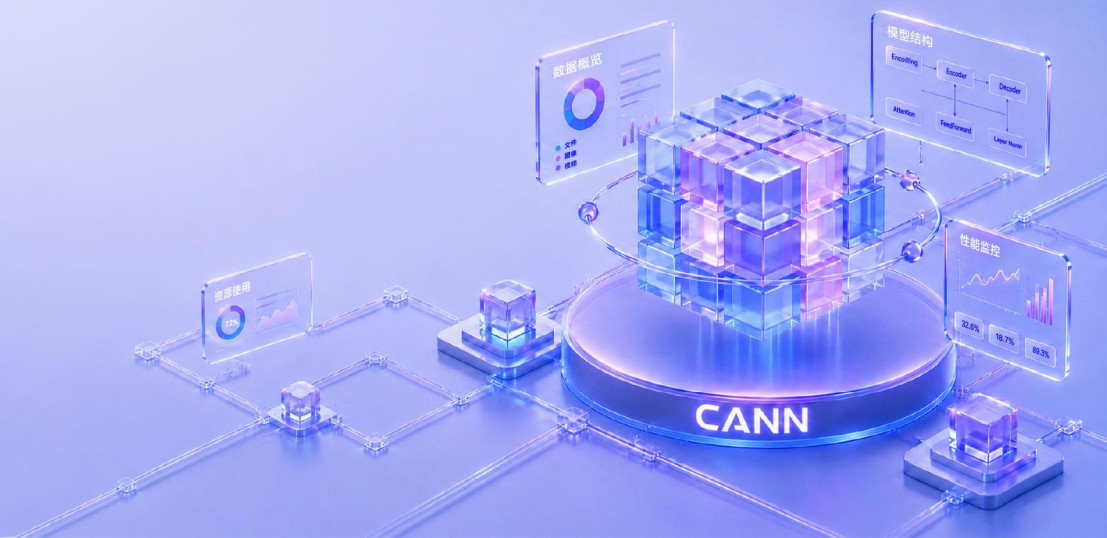




CANN PyPTO编程框架与CANNBot Agent 算子生成工程

——PyPTO KDA落地百灵大模型

潘柏屹



百灵Ling-3.0-flash

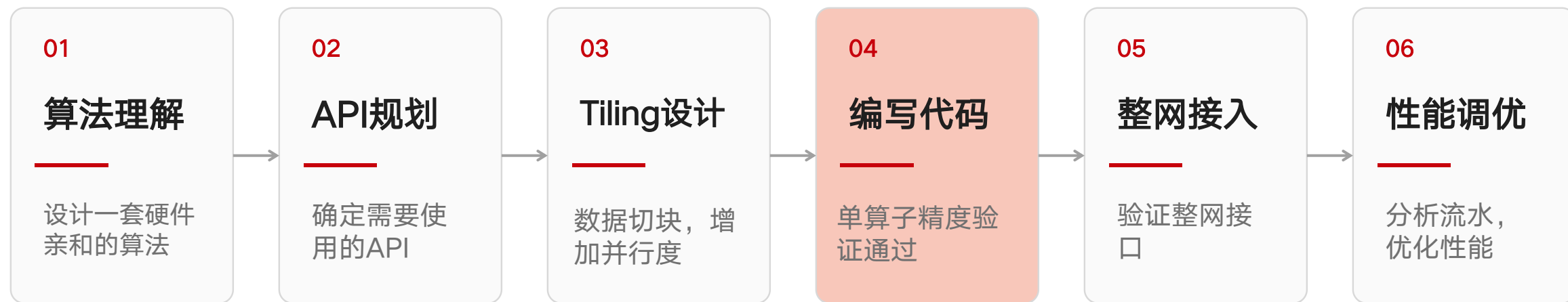
- 使用KDA+MLA的原生混合注意力架构，压缩推理开销
- 在AI Coding，Deep Research等实际工程问题和长上下文场景有着突出表现
- 以极低激活（128B激活5B）参数，平衡开销与性能

Ling-3.0-flash 优势场景与综合指标

Ling-3.0-flash Ring-2.6-IT (xhigh) MiniMax-M2.7 Step-3.7-Flash (high) DeepSeek-V4-Flash (max) Nemotron-3-Super-120B-A12B GPT-5.4-mini (high) Claude-Sonnet-4.6(max)



部署一个高效的融合算子，我们做了什么？



一套包含算子编码在内的完整的工作流

难点1：算子编码难度大

✓ PyPTO提供友好的编程框架

难点2：开发流程复杂

✓ 让Agent来处理工作流

目录

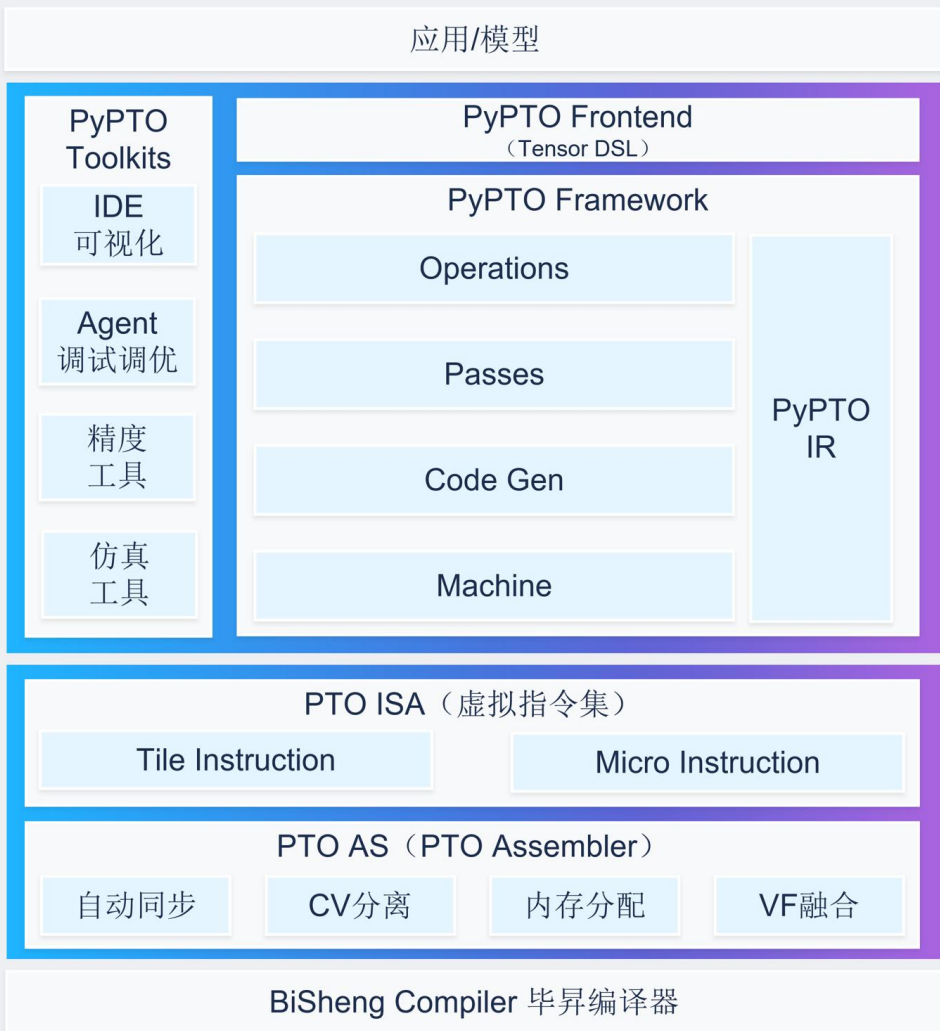
Part 1 CANN PyPTO 编程

Part 2 Agent算子生成框架

Part 3 算子实战：KDA算子

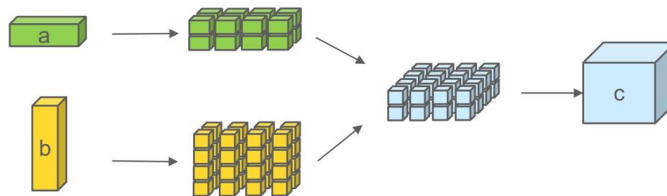
PyPTO: 构建简单易用、性能兼顾的算子编程框架

应用/模型



自动数据切分

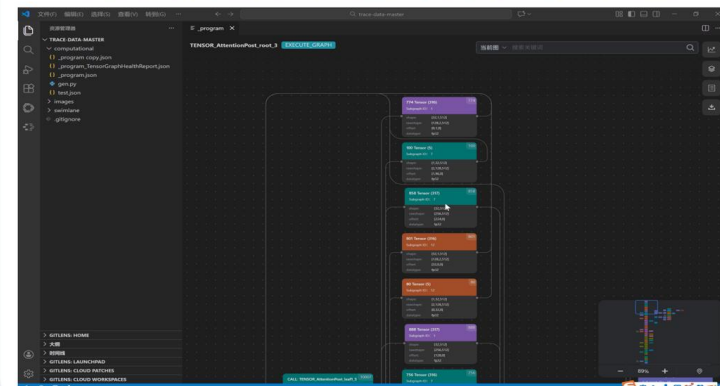
支持自动 **Tile** 切分, 自动适配硬件存储层级
大幅提升编程便捷性



开发者关注于算子 **Tensor** 计算逻辑及计算流表达, 可控制切分粒度,
PyPTO 自动实现 **Tensor** -> **Tile** 的变换, 及运行时数据依赖处理

全调试调优可视化

支持 编译/运行时状态可视化
支持 算子代码-算子计算流程图-算子执行泳道图 实时联动



跨代高效的虚拟指令集底座

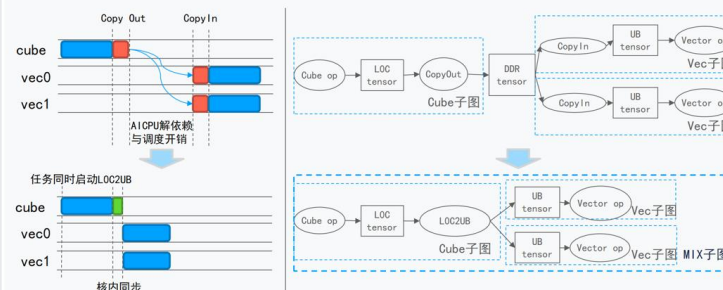
统一指令与内存抽象, 封装**DSA**硬化, 实现跨代兼容不降速



随路量化
转置加速
异步预取
.....

MIX 子图切分, 平滑代际兼容

基于 **950** 新数据通路, 支持自动 **MIX** 子图切分
大幅提升复杂融合算子性能



平衡易用性、高性能、通用性, 打造极致高效编程体验

PyPTO让开发复杂融合算子更简单

PTO = Parallel Tensor/Tile Operation

以Tensor描述“做什么”，以Tile保留“如何高效执行”的优化空间。

- ✓ 易开发：Python友好API，贴近数学表达。让用户关注做什么，交由框架具体去设计怎么做。
- ✓ 易迁移：算法与硬件解耦，快速适配不同硬件平台
- ✓ 符号化编程：内置调优开关，根据不同场景灵活配置
- ✓ 自动Tiling：通过 `set_vec_tile_shapes` / `set_cube_tile_shapes` 设置基本块，框架自动切分Tile感知并行与内存层次
- ✓ MPMD 高效调度：不同核可在同一时间执行不同程序，多核并行，自动负载均衡，不需要手排流水。支持编译、调度、DFX工具链协同

Python表达

```
@pypto.frontend.jit
def fused_op(q, k, v, result):
    qt = pypto.view(q, [x_len, y_len], [x_idx, y_idx])
    kt = pypto.view(k, [x_len, y_len], [x_idx, y_idx])
    vt = pypto.view(v, [x_len, y_len], [x_idx, y_idx])
    score = pypto.matmul(qt, kt)
    value = pypto.matmul(score, vt)
    pypto.assemble(value, [x_idx, y_idx], result)
```



可视化的PyPTO调优工具

The screenshot displays the PyPTO visualization tool interface, which is a web-based IDE for optimizing PyTorch operations. The interface is divided into several panels:

- EXPLORER:** A sidebar on the left showing the file structure of the project. It includes folders like `output`, `src`, and `tests`, and files like `chunk_kda_impl.py`, `chunk_kda_golden.py`, and `chunk_kda_tuning_report.md`.
- Code Editor:** The main area showing the source code of `chunk_kda_impl.py`. The code defines a `chunk_kda_varlen_kernel` function, which is a PyTorch kernel implementation. It includes tensor declarations, memory management, and a loop structure for processing data.
- Search:** A search bar at the top right with the query `test_chunk_kda_varlen, Aa, ab, .*`. It shows "No results" for the current search.
- Terminal/Console:** A panel at the bottom right showing the output of the program. It displays a message: "Operation success."

The interface is running on a remote server, as indicated by the SSH connection information at the bottom: `SSH: 110.239.84.227`.

目录

Part 1 CANN PyPTO 编程

Part 2 Agent算子生成框架

Part 3 算子实战：KDA算子

框架降低编码难度，CANNBot Agent负责开发流程

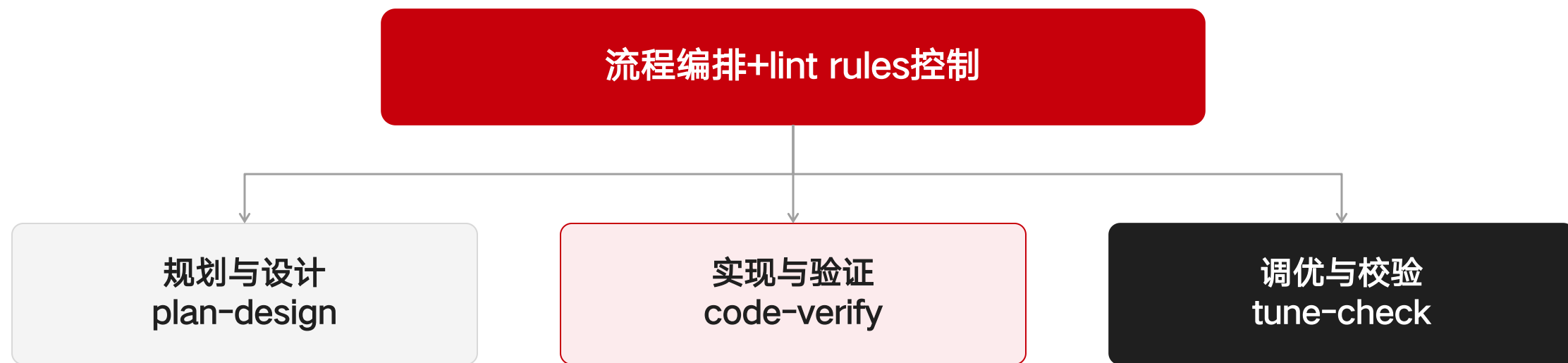
PyPTO已解决：如何用更高层抽象表达可编译、可调优的算子

仍要完成



由Agent来进行节点的推进与具体步骤的执行

CANNBot PyPTO Agent框架设计



Skills | 沉淀专家经验，背靠DFX调试能力

需求与设计

意图理解 · API探索 · 方案设计

实现与验证

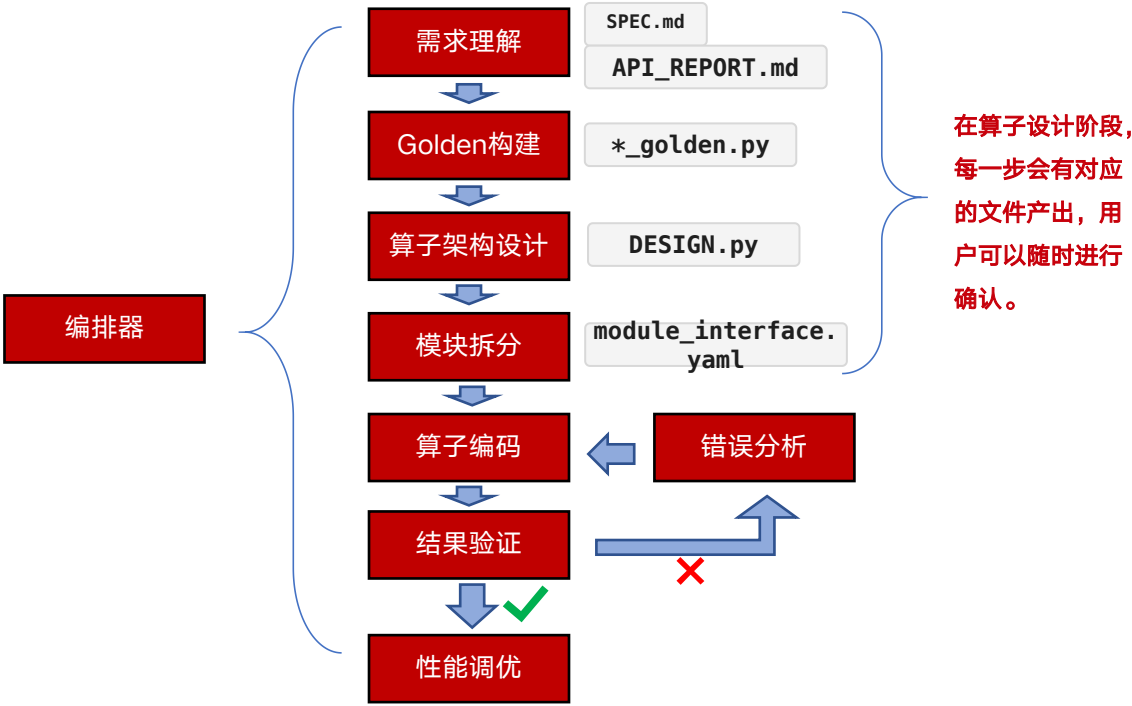
Golden · 代码开发 · 精度诊断

性能调优

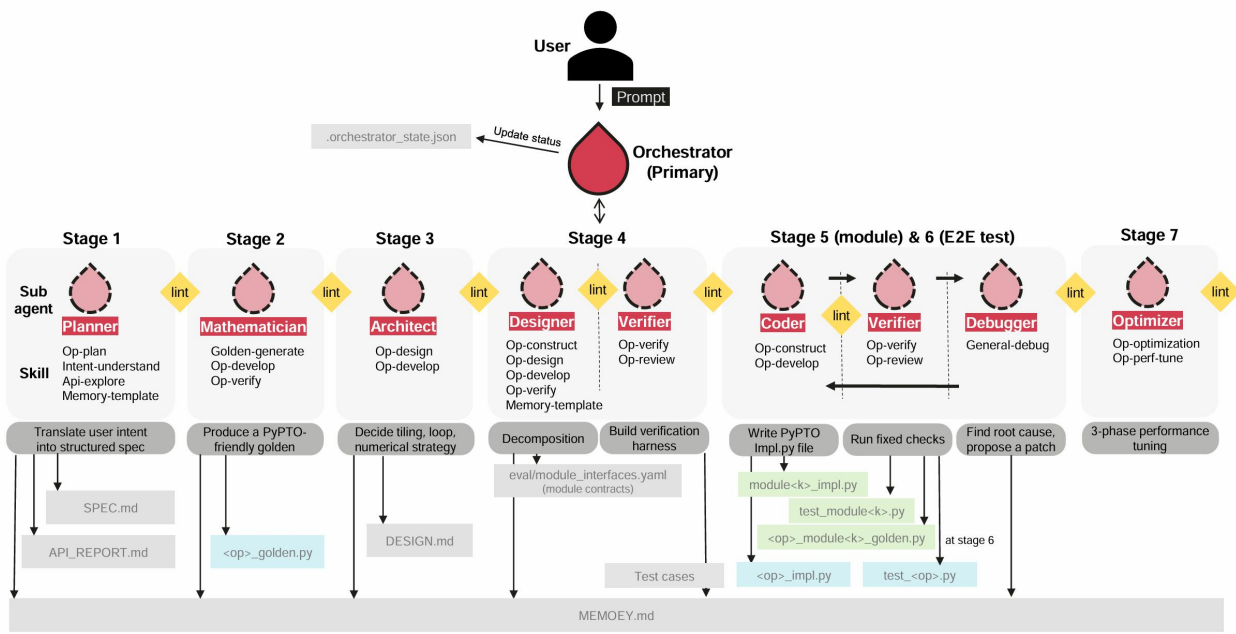
前端结构 · 泳道流水 · 核内配置

CANNBot PyPTO Agent 协作开发

构建端到端的 AI Agent 能力



多智能体协作开发



关键设计:

- 裁判与执行者分离: 验证由独立agent或者lint gate负责执行, 减少agent偷懒作弊的可能
- 递增式分模块开发: 将复杂算子拆分成多个模块, 并采取递增式开发的方式, 减少复杂程度
- subagent与共享记忆: 为了控制上下文大小, 拆分出多个subagent来执行任务, subagent之间无法共享上下文, 通过MEMORY.md来传递信息
- 强制门禁: 在关键节点添加触发式自动门禁, 通过脚本进行产物检查。



目录

Part 1 CANN PyPTO 编程

Part 2 Agent算子生成框架

Part 3 算子实战：KDA算子

一周内完成生成、整网接入、优化与部署



Ling-3.0-flash | 推理前向 | 昇腾A3 | q/k/v BF16 | 最高128K序列

chunk_kda代码样例（PyTorch实现）

```
def naive_chunk_kda(
    q: torch.Tensor,
    k: torch.Tensor,
    v: torch.Tensor,
    g: torch.Tensor,
    beta: torch.Tensor,
    scale: float | None = None,
    initial_state: torch.Tensor | None = None,
    output_final_state: bool = False,
    chunk_size: int = 64,
):
    dtype = v.dtype
    B, T, H, K, HV, V = *q.shape, v.shape[2], v.shape[-1]
    G = HV // H
    BT = chunk_size
    NT = T // BT
    if scale is None:
        scale = K ** -0.5
    assert T % BT == 0

    # Rearrange into chunks: [B, head, NT, BT, ...]
    q, k = [rearrange(x, 'b (n c) h ...-> b h n c ...', c=BT).to(torch.float) for x in [q, k]]
    v, g, beta = [rearrange(x, 'b (n c) h ...-> b h n c ...', c=BT).to(torch.float) for x in [v, g, beta]]
    # Expand q/k to value head dim for GVA: [B, H, ...] -> [B, HV, ...]
    q = q.repeat_interleave(G, dim=1) * scale # [B, HV, NT, BT, K]
    k = k.repeat_interleave(G, dim=1) # [B, HV, NT, BT, K]
    g = g.cumsum(-2)

    # note that diagonal is masked.
    mask = torch.triu(torch.ones(BT, BT, dtype=torch.bool, device=q.device), diagonal=0)

    # Akk uses k (expanded to HV) and g (per value head)
    A = torch.zeros(*g.shape[:-1], BT, dtype=torch.float, device=q.device)
    for i in range(BT):
        k_i = k[:, :, i, :]
        g_i = g[:, :, i:i+1, :]
        A[:, :, i] = torch.einsum('...c d, ...d->...c', k * (g - g_i).exp(), k_i)
    A = A * beta[:, :, None]
```

```
A = -A.masked_fill(mask, 0)
for i in range(1, BT):
    A[:, :, i, :i] = A[:, :, i, :i].clone() + (A[:, :, i, :, None].clone() * A[:, :, :, :i].clone()).sum(-2)
A = (A + torch.eye(BT, dtype=torch.float, device=q.device)) * beta[:, :, None, :]

w = A @ (g.exp() * k)
u = A @ v

S = k.new_zeros(B, HV, K, V).to(q)
if initial_state is not None:
    S += initial_state
o = torch.zeros_like(v)
mask = torch.triu(torch.ones(BT, BT, dtype=torch.bool, device=q.device), diagonal=1)
for i in range(0, NT):
    # [B, HV, BT, ...]
    q_i = q[:, :, i] # [B, HV, BT, K]
    k_i = k[:, :, i] # [B, HV, BT, K]
    u_i = u[:, :, i] # [B, HV, BT, V]
    g_i = g[:, :, i] # [B, HV, BT, K]
    w_i = w[:, :, i] # [B, HV, BT, K]
    # Aqk: per value head (q from qk head, g from value head, k from qk head)
    Aqk = torch.zeros(B, HV, BT, BT, dtype=torch.float, device=q.device)
    for j in range(BT):
        k_j = k[:, :, i, j]
        g_j = g[:, :, i, j:j+1, :]
        Aqk[:, :, j] = torch.einsum('...c d, ...d->...c', q_i * (g_i - g_j).exp(), k_j)
    Aqk = Aqk.masked_fill(mask, 0)
    v_i = u_i - w_i @ S
    o[:, :, i] = (q_i * g_i.exp()) @ S + Aqk @ v_i
    S = S * rearrange(g_i[:, :, -1].exp(), 'b h k-> b h k 1')
    S += rearrange((g_i[:, :, -1:] - g_i).exp() * k_i, 'b h c k-> b h k c') @ v_i
if not output_final_state:
    S = None
return rearrange(o, 'b h n c d-> b (n c) h d').to(dtype), S
```

<https://github.com/fla-org/flash-linear-attention/blob/main/fla/ops/kda/naive.py>

chunk_kda代码样例（PyPTO部分实现）

```
@pypto.frontend.jit(
    runtime_options={"run_mode": pypto.RunMode.NPU, "launch_sched_aicpu_num": 6, "device_sched_mode": 1, "device_sched_parallelism": 4},
    pass_options={
        "vec_nbuffer_setting": {-2: 1, -1: 128},
        "cube_l1_reuse_setting": {-1: 4},
        "cube_nbuffer_setting": {-1: 4}
    },
    debug_options={
        "runtime_debug_mode": 0
    }
)
def chunk_kda_varlen_kernel(
    q: pypto.Tensor([1, pypto.DYNAMIC, pypto.DYNAMIC, 128], pypto.DT_BF16), # [1,T,H,K] packed # SNAP:SIG_JIT
    k: pypto.Tensor([1, pypto.DYNAMIC, pypto.DYNAMIC, 128], pypto.DT_BF16), # [1,T,H,K]
    v: pypto.Tensor([1, pypto.DYNAMIC, pypto.DYNAMIC, 128], pypto.DT_BF16), # [1,T,H,V]
    g: pypto.Tensor([1, pypto.DYNAMIC, pypto.DYNAMIC, 128], pypto.DT_FP32), # [1,T,H,K] fp32
    beta: pypto.Tensor([1, pypto.DYNAMIC, pypto.DYNAMIC], pypto.DT_FP32), # [1,T,H] fp32
    cu: pypto.Tensor([pypto.DYNAMIC], pypto.DT_INT32), # [N+1] segment boundaries (indexed in-kernel)
    tril: pypto.Tensor([128, 128], pypto.DT_FP32), # cumsum incl
    trils: pypto.Tensor([128, 128], pypto.DT_FP32), # strict-lower mask
    eyestk: pypto.Tensor([16, 128], pypto.DT_FP32), # block-inv diag I
    state0: pypto.Tensor([pypto.DYNAMIC, pypto.DYNAMIC, 128, 128], pypto.DT_FP32), # [N,H,V,K] upstream ABI (transpose in-kernel)
    o_out: pypto.Tensor([1, pypto.DYNAMIC, pypto.DYNAMIC, 128], pypto.DT_BF16), # [1,T,H,V]
    s_out: pypto.Tensor([pypto.DYNAMIC, pypto.DYNAMIC, 128, 128], pypto.DT_FP32), # [N,H,V,K] upstream ABI (transpose in-kernel)
    use_qk_l2norm_in_kernel: bool = False, # non-tensor param (after all tensors); trace-time switch
    scale: float = _SCALE, # non-tensor param; trace-time query-scale constant (folded in fp32 in-kernel)
):
    # H/N are derived in-kernel from tensor dims (not int params) so all (H,N) shapes
    # reuse one compile; use_qk_l2norm_in_kernel and scale are the only trace-time
    # (non-tensor) params-> they key the compile cache (scale is realistically constant).
    pypto.experimental.set_operation_options(compile_axis=True) # inline brcb for tail-1 beta broadcasts
    K = q.shape[3] # K == 128
    V = v.shape[3] # V == 128
    H = q.shape[2] # head count, derived from tensor dim (q [1,T,H,K])
    N = state0.shape[0] # seq count, derived from tensor dim (state0 [N,H,V,K])
    # z8/z16/z32: the _inv_merge upper-right zero quadrants ([16,16]/[32,32]/[64,64]),
    # loop-invariant so built once per launch and threaded into _chunk_compute/_inverse8.
    pypto.set_vec_tile_shapes(16, 64)
    z8 = pypto.full(size=[_MIN, _MIN], fill_value=0.0, dtype=pypto.DT_FP32)
    z16 = pypto.full(size=[_MIN * 2, _MIN * 2], fill_value=0.0, dtype=pypto.DT_FP32)
    z32 = pypto.full(size=[_MIN * 4, _MIN * 4], fill_value=0.0, dtype=pypto.DT_FP32)
    # SNAP:before_nt_loop
    for h in pypto.loop(0, H, 1, name="LH", parallel=True): # head loop (independent sessions-> parallel schedule)
        for n in pypto.loop(0, N, 1, name="LN"): # sequence loop
            bos = cu[n] # seq start token (indexed from device cu tensor)
            eos = cu[n + 1] # seq end token (exclusive)
            L = eos - bos # seq length (SymbolicScalar; may be %128 != 0)
            pypto.set_cube_tile_shapes([128, 128], [128, 128], [128, 128])
            pypto.set_vec_tile_shapes(1, 1, 128, 128) # rank-4 for state0[N,H,V,K] view (upstream ABI)
            s_seed_vk = pypto.reshape(pypto.view(state0, [1, 1, V, K], [n, h, 0, 0]), [V, K]) # [V,K] upstream ABI slice
            pypto.set_vec_tile_shapes(128, 128) # rank-2 [V,K]->[K,V] transpose (V==K==128)
            s_carry = s_seed_vk.transpose(0, 1) + 0.0 # [V,K]->[K,V] internal seed (transpose in-kernel)
            pypto.set_vec_tile_shapes(128, 128) # back to rank-2
            # dynamic-bound static-step chunk loop (step_BT); last iteration is the
            # possibly partial tail
```

```
for s_idx in pypto.loop(0, L, _BT, name="LNT", submit_before_loop=False, unroll_list=[8]):
    # SNAP:inside_nt_loop
    off = bos + s_idx # absolute token offset of this chunk
    actual_l = (L - s_idx).min(_BT) # valid rows in THIS chunk (<=_BT); tail may be < _BT
    #----rank-4 strided-head-slice-views; valid_shape marks the actual rows.
    #----Cube tile for the M1/M2 matmuls is set inside _chunk_compute.----
    pypto.set_vec_tile_shapes(1, 128, 1, 128) # rank-4 q/k/v/g chunk views
    q4 = pypto.view(q, [1, _BT, 1, K], [0, off, h, 0], valid_shape=[1, actual_l, 1, K])
    k4 = pypto.view(k, [1, _BT, 1, K], [0, off, h, 0], valid_shape=[1, actual_l, 1, K])
    v4 = pypto.view(v, [1, _BT, 1, V], [0, off, h, 0], valid_shape=[1, actual_l, 1, V])
    g4 = pypto.view(g, [1, _BT, 1, K], [0, off, h, 0], valid_shape=[1, actual_l, 1, K])
    q2 = pypto.reshape(q4, [_BT, K], valid_shape=[actual_l, K]) # [128,K], rows=actual_l are pad
    k2 = pypto.reshape(k4, [_BT, K], valid_shape=[actual_l, K])
    v2 = pypto.reshape(v4, [_BT, V], valid_shape=[actual_l, V])
    g2 = pypto.reshape(g4, [_BT, K], valid_shape=[actual_l, K])
    pypto.set_vec_tile_shapes(1, 128) # rank-2 beta view/reshape
    b3 = pypto.view(beta, [1, _BT, 1], [0, off, h], valid_shape=[1, actual_l, 1])
    b2 = pypto.reshape(b3, [1, _BT], valid_shape=[1, actual_l]) # [1,128], cols=actual_l are pad
    #----tail branch: fillpad zeros the pad region of the last, possibly-
    #----partial chunk; full chunks (actual_l==_BT) skip fillpad.----
    if pypto.is_loop_end(s_idx):
        pypto.set_vec_tile_shapes(128, 128) # q2/k2/v2/g2 are [128,128]
        qc = pypto.fillpad(q2, "constant", 0.0) # rows [actual_l:_BT]-> 0.0
        kc = pypto.fillpad(k2, "constant", 0.0)
        vc = pypto.fillpad(v2, "constant", 0.0)
        gc = pypto.fillpad(g2, "constant", 0.0) # KEY: g pad=0-> gcum flat (glast valid)
        pypto.set_vec_tile_shapes(1, 128) # b2 is [1,128]
        bc = pypto.fillpad(b2, "constant", 0.0) # cols [actual_l:_BT]-> 0.0
        oc, s_new = _chunk_compute(qc, kc, vc, gc, bc, s_carry, tril, trils, eyestk,
                                   use_qk_l2norm_in_kernel, scale, z8, z16, z32)
        s_carry[:] = s_new # carry state across chunks
        pypto.set_vec_tile_shapes(1, 128, 1, 128) # rank-4 o reshape+assemble
        o_chunk = pypto.reshape(oc, [1, _BT, 1, V])
        pypto.assemble(o_chunk, [0, off, h, 0], o_out) # only actual_l rows valid (oc already bf16)
        # writes o_out[0, off:off+actual_l, h, :]
    else:
        oc, s_new = _chunk_compute(q2, k2, v2, g2, b2, s_carry, tril, trils, eyestk,
                                   use_qk_l2norm_in_kernel, scale, z8, z16, z32)
        s_carry[:] = s_new
        pypto.set_vec_tile_shapes(1, 128, 1, 128) # rank-4 o reshape+assemble
        o_chunk = pypto.reshape(oc, [1, _BT, 1, V]) # full 128 rows (oc already bf16)
        pypto.assemble(o_chunk, [0, off, h, 0], o_out) # non-tail chunk is always full
    pypto.set_vec_tile_shapes(128, 128)
    pypto.set_vec_tile_shapes(128, 128) # rank-2 [K,V]->[V,K] transpose
    s_carry_vk = s_carry.transpose(0, 1) # [K,V]->[V,K] upstream ABI (transpose IN-KERNEL)
    pypto.set_vec_tile_shapes(1, 1, 128, 128) # rank-4 reshape+assemble
    s_chunk = pypto.reshape(s_carry_vk, [1, 1, V, K]) # [N,H,V,K] upstream ABI
    pypto.assemble(s_chunk, [n, h, 0, 0], s_out)
```

把精力放在算法与边界，让框架和Agent承担重复工程

- PyPTO降低算子的表达门槛
- Agent压缩从算法到部署的工程路径

01

从明确边界开始

Shape、dtype、部署范围、性能口径

02

用产物链推进

SPEC → Golden → DESIGN → 实现

03

用门禁决定部署

精度、性能与安全审查缺一不可

目标：让融合算子容易开发，容易部署，容易迁移

CANN PYPTO开发社区

<https://gitcode.com/cann/cannbot-skills> : CANNBot Agent

<https://gitcode.com/cann/pypto>: PyPTO编程框架

<https://gitcode.com/cann/pypto-gym>: PyPTO算子库

<https://gitcode.com/cann/pto-isa>: PTO-ISA虚拟指令集

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

